

LISTA ENLAZADA SIMPLE

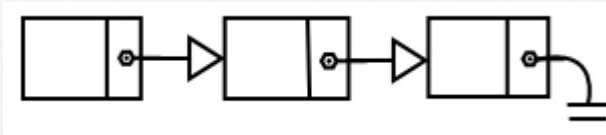
ESTRUCTURA DE DATOS Y ALGORITMOS

INF - 131

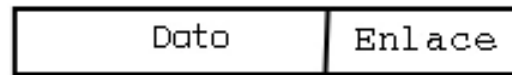
Lic. Marcelo Aruquipa

LISTAS ENLAZADAS

- Es un conjunto de nodos los cuales almacenan datos, información, objetos ,etc.



- **Estructura.** Un nodo tiene la siguiente estructura



Donde:

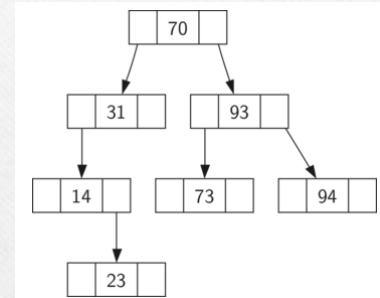
- **Dato:** es el ítem o elemento que va a contener
 - **Enlace:** es un puntero al siguiente elemento de la lista
- **Acceso.** El acceso a la lista enlazada es por medio de la dirección que se almacena en el nodo denominado **Enlace**.

Lic. Marcelo Aruquipa

LISTAS ENLAZADAS

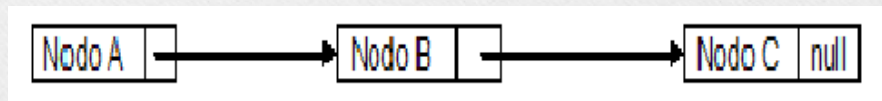
- **Aplicaciones.**

- Fila de supermercado, calendario diario, juegos, transacciones bancarias, canales de televisión , presentaciones de diapositivas, citas medicas de pacientes, implementación de arboles, etc.

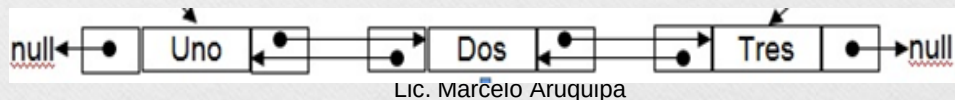


- **Tipos de listas.** Existen 3 tipos de listas:

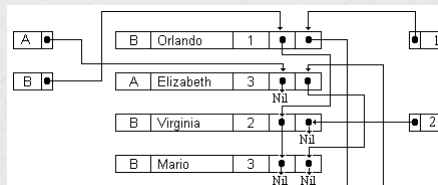
- Listas simples



- Listas dobles



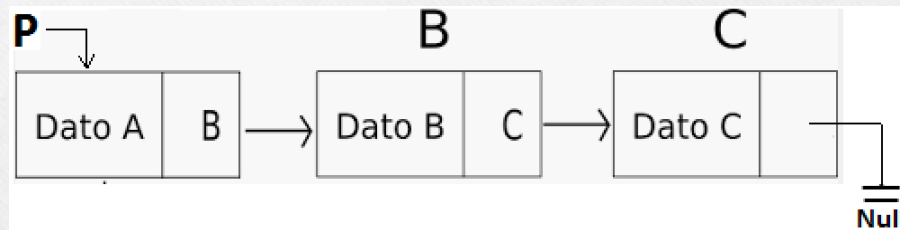
- Listas múltiples



LISTAS SIMPLE NORMAL

- Es una estructura lineal dinámica.
- Sus enlaces son direcciones a otros nodos.

Gráficamente. Una lista simple normal se representa:



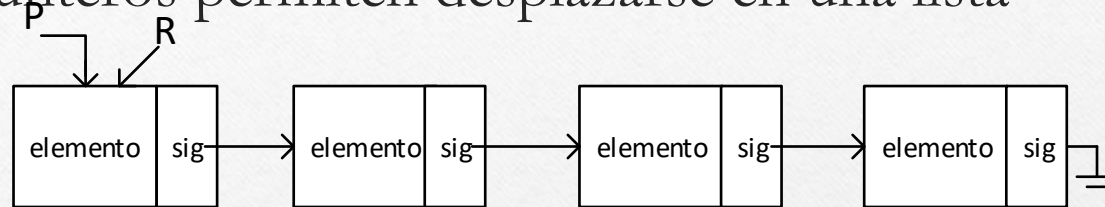
Donde:

- **P** denominado Nodo Raíz.
- **Null** nodo nulo, final de la lista.

Lic. Marcelo Aruquipa

LISTAS SIMPLE NORMAL

- Los punteros permiten desplazarse en una lista



Donde por notación:

P : Es el Puntero Raíz de la lista

R: Es un Puntero Auxiliar

elemento: Dato que almacena el nodo

sig: Puntero que tiene la dirección del siguiente **Nodo**

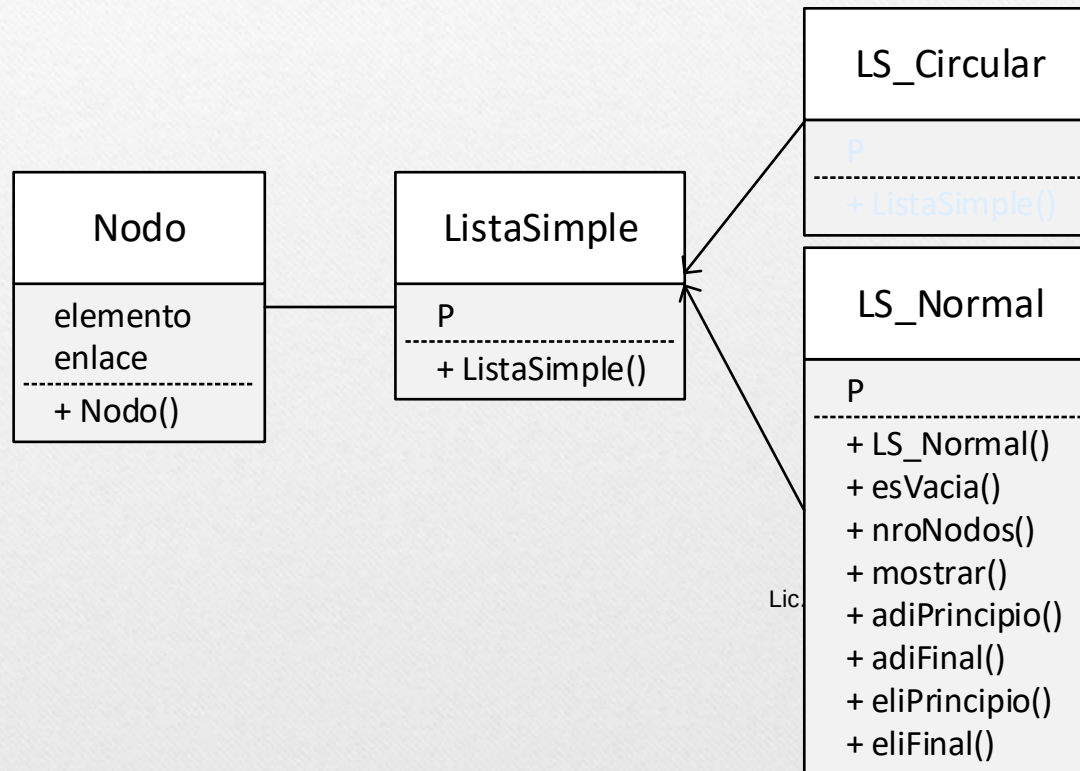
Entonces **P** tiene la dirección del inicio de la lista(apunta al principio al primer **Nodo**).

El puntero **R** es el que se utilizara para desplazarse en la lista

Lic. Marcelo Aruquipa

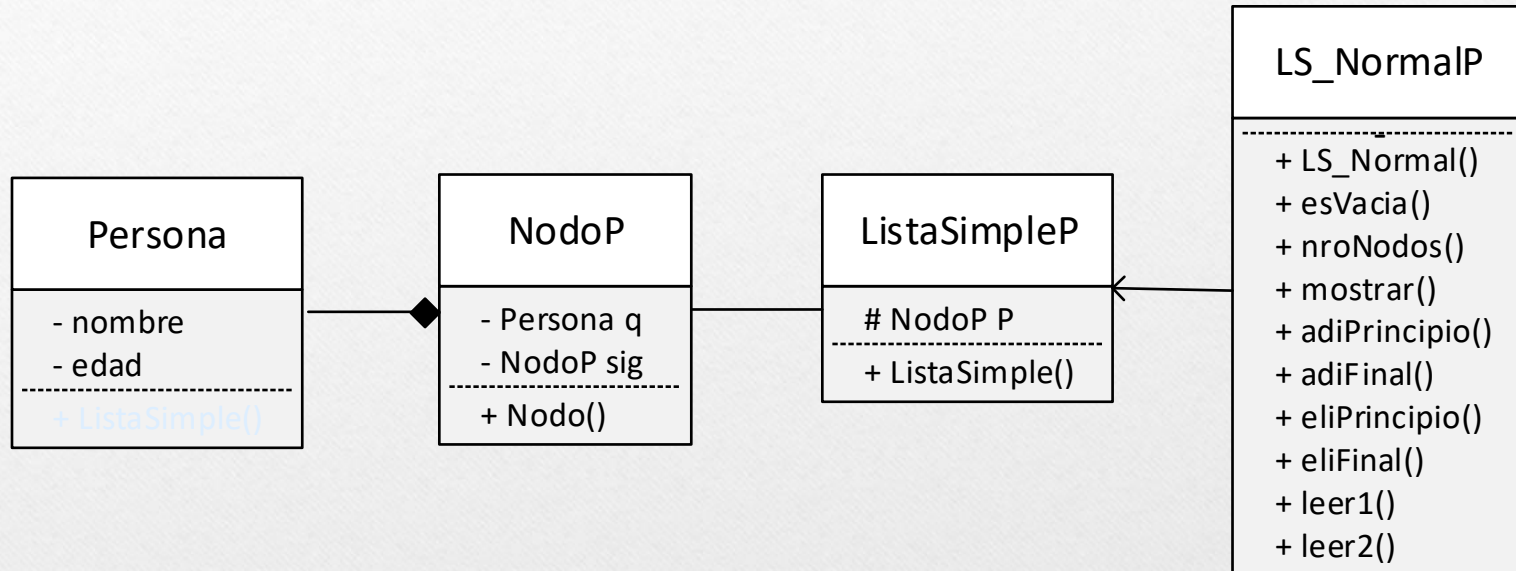
LISTAS SIMPLE NORMAL

Diagrama de Clases



LISTAS SIMPLE NORMAL

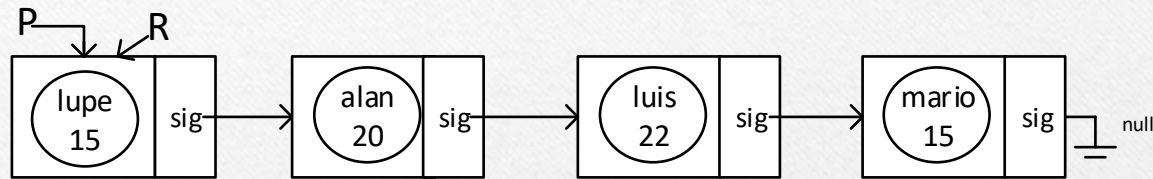
Ejemplo. Implementar una lista de objetos Persona.



Lic. Marcelo Aruquipa

LISTAS SIMPLE NORMAL

Si tenemos una lista de objetos persona (nombre y edad)



Donde por notación:

P : Es el Puntero Raíz de la lista

R: Es un Puntero Auxiliar

elemento: Objeto persona que almacena el nodo

Lic. Marcelo Aruquipa

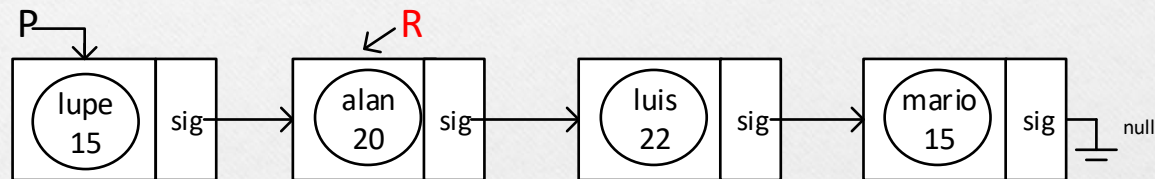
sig: Puntero que tiene la dirección del siguiente **Nodo**

LISTAS SIMPLE NORMAL

Avanzar: para que R avance (apunte al siguiente nodo) se debe hacer:

$$R \leftarrow R.\text{sig}$$

*“R tiene la dirección del siguiente **Nodo**”* ó *“R apunta al siguiente **Nodo**”*

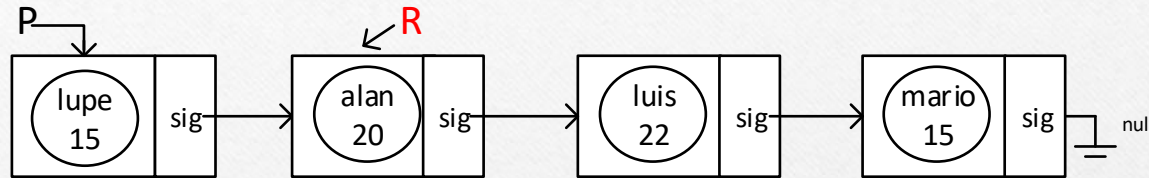


Esto se puede realizar mientras R sea distinto de null

```
while(R != null)
{
    R <- R.getSig();
}
```

Lic. Marcelo Aruquipa

LISTAS SIMPLE NORMAL



Cuando R apunta al siguiente Nodo, se puede acceder al elemento del nodo para **obtener** el elemento o para **modificarlo**

var $\leftarrow$ R.elemento;

Ó

R.Elemento $\leftarrow$ dato;

```
R.getQ().mostrar();
```

```
R.getQ().getEdad();
```

Lic. Marcelo Aruquipa

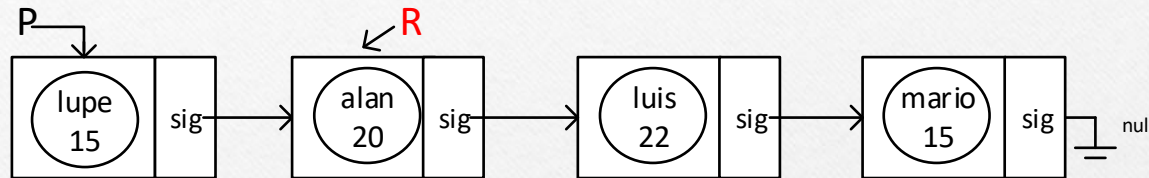
```
R.setSig(null);
```

```
R.setSig(P);
```

```
R.setSig(R.getSig());
```


LISTAS SIMPLE NORMAL

Recorrer una lista



Observaciones:

- ✓ Hacer notar que el avance en la lista es en una sola dirección (derecho), es decir que no se puede retroceder.
- ✓ Se puede utilizar mas de un puntero auxiliar.
- ✓ Si el puntero Raiz (**P**) se pierde se perderá toda la lista.
- ✓ Al principio **R** siempre apuntara a la **Raiz P**, para luego desplazarse

Lic. Marcelo Aruquipa

LISTAS SIMPLE NORMAL

Clase Nodo

```
class NodoP
{
  private:
    Persona q;
    NodoP sig;
  public:
    NodoP()
    {
      sig <- null;
    }
}
```

Clase ListaSimpleP

```
class ListaSimpleP
{
  protected:
    NodoP P;
  public:
    ListaSimpleP()
    {
      P <- null;
    }
}
```

Se asume la implementación de la Lic. Marcelo Aruquipa **clase Persona** (atributos y metodos)

LISTAS SIMPLE NORMAL

Clase LS_NormalP

```
class LS_NormalP : ListaSimpleP
{
    public:
        bool esVacia()
        {
            if(P == null)
                return true;
            return false;
        }
        int nroNodos()
        {
            int c <- 0;
            if(esVacia())
                return 0;
            else{
                NodoP R <- P;
                while(R != null)
                {
                    c++;
                    R <- R.getSig();
                }
            }
            return c;
        }
}
```

```
mostrar()
{
    NodoP R <- P;
    while(R != null)
    {
        R.getQ().mostrar();
        R <- R.getSig();
    }
}

adiPrincipio(Persona z)
{
    nuevo <- new NodoP();
    nuevo.setQ(z);
    nuevo.setSig(P);
    P <- nuevo;
}
```

```
adiFinal(Persona z){
    nuevo <- new NodoP();
    nuevo.setQ(z);
    if(esVacia())
        P <- nuevo;
    else{
        NodoP R <- P;
        while(R.getSig() != null)
        {
            R <- R.getSig();
        }
        R.setSig(nuevo);
    }
}
```

Lic. Marcelo Aruquipa

LISTAS SIMPLE NORMAL

Clase LS_NormalP

```
NodoP eliFinal()
{
    x <- new NodoP();
    if(!esVacia())
    {
        if(nroNodos() == 1)
        {
            x <- P;
            P <- null;
        }
        else{
            NodoP R <- P;
            NodoP S <- P;
            while( R.getSig() != null)
            {
                S <- R;
                R <- R.getSig();
            }
            S.setSig(null);
            x <- R;
        }
    }
    return x;
}
```

```
NodoP eliPrincipio()
{
    x <- new NodoP();
    if(!esVacia())
    {
        x <- P;
        P <- P.getSig();
        x.setSig(null);
    }
    return x;
}
```

Lic. Marcelo Aruquipa

LISTAS SIMPLE NORMAL

Clase LS_NormalP

```
leer1(int n)
{
    for (i <- 1 to n)
    {
        z <- new Persona();
        z.leer();
        adiPrincipio(z);
    }
}

leer2(int n)
{
    for (i <- 1 to n)
    {
        z <- new Persona();
        z.leer();
        adiFinal(z);
    }
}

} //fin clase LS_NormalP
```

```
main()
{
    A <- new LS_NormalP();
    A.leer1(5);
    A.mostrar();

    Persona z <- A.eliPrincipio();
    z.mostrar();

    A.adiUltimo(z);
    A.mostrar();
}
```

Lic. Marcelo Aruquipa

Preguntas??
